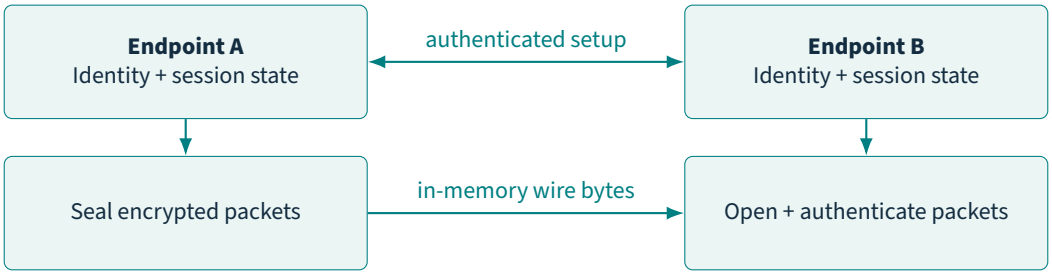
Session establishment and per-packet processing

The two experiments examine complementary prerequisites for offloading PRP processing. EXP-001 asks whether session establishment can be separated from subsequent packet processing. EXP-002 asks whether an aggregator can remove one routing layer and forward retained data without copying the body at four specific software operations. Neither experiment demonstrates a hardware accelerator.

Question and execution environment

After an authenticated session has been established, must each received packet repeat relationship policy, handshake or key derivation? EXP-001 runs two real **libprp** endpoints with separate identities and session handles in one process. They exchange authenticated encrypted packets through in-memory byte buffers. There are no sockets, kernel modules or VMs in this experiment.



What is observed

The reproduction builds the bundled, pinned library source with native function-entry instrumentation. Observations distinguish the setup phase from the receive phase; they do not replace the library's cryptographic operations with a mock.

Setup work	Relationship-policy evaluation, authenticated negotiation and session-key derivation.
Packet work	Dispatch using installed session state, packet opening/authentication, replay protection and applicable state checks.

Procedure and controls

1. Establish: create the endpoints and complete authenticated setup.

2. Exercise: run workloads of 0, 1, 10, 1,000 and 100,000 packets, observing setup and receive separately.

3. Challenge: use deliberate regressions and lifecycle/adversarial cases to check that the observer detects forbidden repeated work and that required checks remain active.

4. Verify: validate observations and pinned inputs; print a conclusion and save the detailed evidence. Failed or incomplete runs do not establish the result.

Conclusion supported by the experiment

In the tested sessions, packet processing reuses installed session state without repeating relationship policy, handshake or key derivation. This separation makes a bounded packet-processing path plausible for device offload. **Per-packet authentication and state validation still remain necessary.**

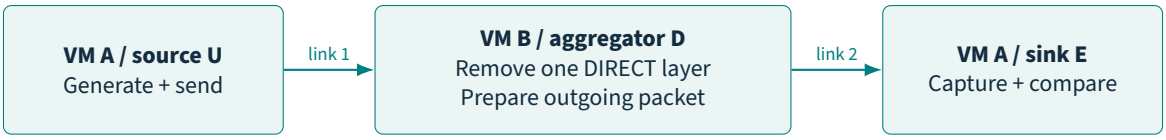
This is an instrumented functional experiment, not a throughput benchmark, a proof for every possible session lifecycle, or a measurement of CPU savings. Its encrypted user-space sessions are distinct from EXP-002's kernel forwarding with the NONE suite.

Reproduction: `sudo ./run` executes both experiments and verifies their results. A successful complete cycle removes its own downloads and evidence; failures preserve diagnostics. Separate run/verify commands retain evidence for inspection. Numerical results are reported by the test, not fixed in this note.

Aggregator forwarding and buffer reuse

The experiment examines whether a PRP aggregator can remove one routing layer and prepare the outgoing packet using existing memory segments, rather than copying its complete body into another assembly buffer. CPU offload means moving work from the main CPU to a device. This test examines a prerequisite for that design, not the device itself; lower elapsed time is not required.

Topology and packet path



The source and sink are separate endpoints in VM A; the aggregator runs in VM B. Two isolated virtual Ethernet links carry the packets. Both guests use one read-only base with private disk overlays; they do not share PRP payload memory. No physical network accelerator is part of this topology.

Protocol elements used in the test

ONION identifies a nested routing representation. Its outer **DIRECT** layer selects the next output and contains a child layer to forward. Removing the outer layer leaves that child intact. The aggregator does not execute the child in this hop.

The **annex** is the attached data region: in this fixture, a short prefix and a deterministic payload. The selected **NONE** session suite does not encrypt the data. Session setup remains authenticated. Annex commitments remain in the packet, but eligible transit does not recompute their hashes in the measured path. Full-byte verification is performed afterwards. NONE isolates forwarding because this lab has no cryptographic accelerator; extending the offload benefit to encrypted suites would require compatible cryptographic hardware, a hypothesis not tested here.

Reference-based forwarding

The current profile uses one **REFERENCE** path: the aggregator prepares new framing and references retained child/annex segments. Their backing storage remains owned while it is needed. No COPY comparison is executed. References are internal memory descriptors; the actual packet bytes still travel over each virtual Ethernet link.

Four kernel counters account for logical copied bytes at **ingress, DIRECT processing, output preparation and carrier submission**. The reader takes before/after snapshots and checks their differences. Zero means those instrumented operations did not copy bytes; it does not mean the CPU made no reads or that every layer of the system is copy-free.

Procedure and interpretation

1. Prepare: establish the sessions and route; generate the input and expected output independently. Payload bytes repeat 00–FF.

2. Forward: forward 32 warmup and 32 measured packets with a 1,024-byte payload. Independently capture input and output on the isolated links.

3. Verify: compare complete delivered bytes with the oracle, bind records to their context and inspect the four copy counters and retained-buffer lifecycle counters. An acknowledgement alone is insufficient.

4. Interpret: preserved delivery plus zero deltas at the four copy operations supports this reference-forwarding mechanism. It does not establish NIC offload, CPU savings or physical memory traffic. Such claims require their own measurements and hardware.

Startup may take several minutes while both guests boot and report their kernels. Those observation times include startup and are not forwarding measurements. Delivery and copy accounting are the result; elapsed time is descriptive, not a statistical speedup claim.

PRP layer removal and wire representation

Byte ranges below are inclusive, relative to the PRP unit (Ethernet payload). Multi-byte PRP integers are big-endian. $N = 1024$. Kind 2 means ONION; it is **not** the session cipher-suite field.



Diagram notation. B means bytes; N is the payload length. "Header" groups the 8-byte PRP record prefix and the 2-byte layer length (offsets 0–9), not the Ethernet header. The new header is also 10 bytes: the route alias and lengths change. Arrows indicate unchanged byte ranges. Block widths are schematic.

Input offsets	Output offsets	Bytes / meaning
0–1	0–1	2: record count = 1
2–3	2–3	2: route alias 1 → 2100 (0x0834)
4; 5	4; 5	1 + 1: kind = 2; flags = 0
6–7	6–7	2: body length $N + 100 \rightarrow N + 58$
8–9	8–9	2: DIRECT layer length $84 \rightarrow 42$
10–11	removed	2: outer commitment length = 32
12–43	removed	32: SHA256(annex), precomputed before timing
44–45	removed	2: outer branch count = 1
46–49	removed	4: egress capability = 100
50–51	removed	2: child length = 42
52–53	10–11	2: child commitment length = 32
54–85	12–43	32: child SHA256(annex), unchanged
86–87	44–45	2: child branch count = 1
88–91	46–49	4: child capability = 0xFFFFFFFF
92–93	50–51	2: child length = 0; aggregator does not execute this child
94–95; 96–97	52–53; 54–55	2 + 2: selected annex prefix fields = 17, 3
98–105	56–63	8: sequence = $30 + i$, $i = 0 \ldots 63$ per phase
106–107	64–65	2: payload length = N
108–$(N + 107)$	66–$(N + 65)$	N: payload byte $j = j \bmod 256$

Ethernet and local submission envelopes

Ethernet (both links). Bytes 0–5 destination MAC; 6–11 source MAC; 12–13 EtherType `88 FB`; PRP starts at 14. MACs change between links. Captured frame lengths are $N + 122$ in / $N + 80$ out. Captures exclude preamble and FCS.

Netlink submission (tested x86-64 ABI, little-endian). 0–3 length $N + 144$; 4–5 discovered family; 6–7 flags 5; 8–11 sequence; 12–15 port. 16–19: command19/version1/reserved0. 20–23: attribute length12/type23; 24–31: peer64. 32–35: attribute length$N + 112$/type32; 36 onward: input PRP. **ACK36**: 0–15 Netlink header (type2); 16–19 signed error0; 20–35 original request header. An ACK alone is not delivery evidence.

Scope: the selected single-target NONE diagnostic profile, not a general protocol specification. Signed setup/HS2 and other suites are outside this data-wire atlas. Full captured request/ACK/frame bytes and receipt digests are retained with runs outside the automatic cleanup cycle. Integrity hashes bind files; they do not independently authenticate native provenance. No claim of hardware acceleration, complete routing cost or physical zero-copy.